SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Error Analysis Task

Code of Conduct:

I Soorya S / 192311060 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SooryaS

1. Online Shopping System – Equivalence Partitioning and Boundary Value Analysis

Question:

A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges, mixing valid and invalid inputs in one class. Boundary values are also not considered. Analyze the errors, identify their root causes, propose corrected test cases, and justify how the corrections improve defect detection.

Answer:

1. Understanding the System

An online shopping system accepts product quantities, prices, discount codes, and other user inputs. Equivalence Class Partitioning (ECP) divides inputs into groups expected to behave similarly, while Boundary Value Analysis (BVA) tests values near the limits.

2. System Decomposition

- Input Validation: Checks quantity, price, and discount code.
- Cart Management: Adds and removes products.
- Price Calculation: Calculates subtotal and discounts.
- Order Processing: Confirms the purchase.
- Error Handling: Displays messages for invalid inputs.

3. Errors and Root Causes

- Valid and invalid inputs are grouped together.
- Minimum and maximum limits are not identified.
- Values immediately below and above the limits are ignored.
- Random test values may not expose boundary-related defects.
- Expected results are not clearly defined for each class.

For example, if the permitted quantity is 1 to 100, grouping all positive values together incorrectly treats 101 as valid.

4. Corrected Equivalence Classes

Assume the permitted product quantity is 1 to 100.

Class	Input	Expected Result
EC1	Quantity less than 1	Reject
EC2	Quantity from 1 to 100	Accept
EC3	Quantity greater than 100	Reject
EC4	Non-numeric quantity	Display validation error

EC5	Empty quantity	Display validation error

5. Boundary Value Test Cases

Test Case	Quantity	Expected Result
TC1	0	Reject
TC2	1	Accept
TC3	2	Accept
TC4	99	Accept
TC5	100	Accept
TC6	101	Reject

6. Improved Testing Strategy

Apply ECP to select representative values from valid and invalid classes. Apply BVA to test values at and around the minimum and maximum limits. Also test invalid formats, empty inputs, and discount-code errors.

Conclusion: Correct ECP classification prevents valid and invalid inputs from being mixed. BVA detects errors near limits, improving test coverage and the reliability of the shopping system.

---

2. Banking Application – Evaluation of Random Testing

Question:

A QA engineer performs random testing on a banking application and claims high coverage without using structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the approach, identify root causes, and recommend appropriate black box testing techniques.

Answer:

1. Understanding the System

A banking application processes deposits, withdrawals, transfers, and balance enquiries. Transactions must follow amount limits, balance rules, and account restrictions to prevent financial errors.

2. System Decomposition

- Authentication: Verifies customer identity.
- Account Validation: Checks account status and details.
- Transaction Validation: Checks amount and transaction limits.
- Balance Management: Updates account balances.

- Transaction Processing: Executes transfers and withdrawals.
- Error Handling: Rejects invalid or failed transactions.

3. Errors in the Existing Approach

- Random inputs may not cover important limits.
- High numbers of tests do not necessarily mean high coverage.
- Important valid and invalid input classes may be missed.
- Expected results and coverage criteria may not be defined.
- Repeated random tests may execute similar scenarios without testing new conditions.

4. Root Causes

The main cause is the absence of a structured test design. Random testing depends on the generated values and may not include exact boundary values or important combinations of conditions.

5. Corrected Test Cases

Assume a withdrawal limit of ₹50,000 per transaction and an available balance of ₹60,000.

Test Case	Withdrawal Amount	Expected Result
TC1	₹0	Reject
TC2	₹1	Accept if permitted by bank rules
TC3	₹49,999	Accept if other conditions pass
TC4	₹50,000	Accept if other conditions pass
TC5	₹50,001	Reject
TC6	₹60,001	Reject due to insufficient balance
TC7	Negative amount	Reject
TC8	Non-numeric amount	Display validation error

6. Improved Testing Strategy

- Use ECP to group valid and invalid transaction amounts.
- Use BVA to test minimum and maximum transaction limits.
- Use Decision Table Testing for combinations of account status, balance, and transaction limits.
- Use State Transition Testing to validate transaction states.
- Use random testing as an additional technique for discovering unexpected defects.

Conclusion: Random testing alone cannot guarantee adequate coverage. Combining structured black box techniques with random testing improves defect detection, transaction reliability, and financial accuracy.

3. White Box Testing – Incomplete Control Flow Graph

Question:

A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors, identify the root causes, reconstruct a proper control flow representation, and explain how improved path testing detects defects.

Answer:

1. Understanding the System

White box testing examines the internal structure of a program. A Control Flow Graph (CFG) represents statements, decisions, loops, and possible execution paths. Missing nodes or edges can leave important logic untested.

2. System Decomposition

- Input Processing: Receives and validates input.
- Decision Logic: Evaluates conditions.
- Loop Control: Repeats statements while a condition holds.
- Processing Logic: Performs calculations or operations.
- Output Handling: Returns results or error messages.

3. Sample Program

```
def process_numbers(n):
    if n > 0:
        i = 0
        while i < n:
            if i % 2 == 0:
                print("Even")
            else:
                print("Odd")
            i += 1
    else:
        print("Invalid input")
```

4. Corrected Control Flow Representation

Start → Check whether n > 0.

- If false, display "Invalid input" and terminate.
- If true, initialize i = 0.
- Check whether i < n.
- If false, terminate.

- If true, check whether i is even.
- If even, display "Even".
- Otherwise, display "Odd".
- Increment i and return to the loop condition.

The CFG must include both outcomes of each decision and the loop-back edge.

5. Independent Execution Paths

Path	Input	Execution
P1	n = 0	Invalid input
P2	n = 1	One even iteration
P3	n = 2	Even, then odd
P4	n = 3	Even, odd, even

These tests cover the invalid-input branch, loop entry, loop termination, and both outcomes of the inner decision. Additional tests can be used for larger loop counts.

6. Errors and Root Causes

- Missing decision nodes hide alternative outcomes.
- Missing loop edges prevent repeated execution from being represented.
- Untested paths can contain calculation and termination errors.
- Statement coverage may be mistaken for complete path coverage.

7. Improved Testing Strategy

Construct the CFG directly from the program logic. Identify all decisions and loops, calculate cyclomatic complexity, and select independent paths. Test zero iterations, one iteration, multiple iterations, and both outcomes of each nested decision.

Conclusion: A complete CFG provides a reliable basis for path testing. Testing independent paths improves logic coverage and helps detect errors in decisions, loops, and termination conditions.

---

4. Statement Coverage – Branch and Path Coverage

Question:

A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of testing, identify coverage gaps, explain why statement coverage is insufficient, and recommend improved white box testing strategies.

Answer:

1. Understanding the System

Statement coverage measures whether every executable statement has run at least once. However, executing every statement does not guarantee that every decision outcome or condition has been tested.

2. System Decomposition

- Input Module: Accepts values.
- Decision Module: Evaluates conditions.
- Processing Module: Performs calculations.
- Output Module: Returns results.
- Error Handling Module: Handles invalid inputs.

3. Example Program

```
def check_age(age):
    if age >= 18:
        print("Eligible")
    else:
        print("Not eligible")
```

4. Test Coverage Analysis

Test Case	Input	Expected Output	Branch
TC1	20	Eligible	True
TC2	16	Not eligible	False
TC3	18	Eligible	True
TC4	17	Not eligible	False

Testing only TC1 and TC2 executes both statements and achieves 100% statement coverage. It also achieves branch coverage for this simple example, but it does not test the exact age boundary. TC3 and TC4 verify behavior immediately around that boundary.

5. Limitations of Statement Coverage

- It does not always execute every decision outcome.
- It may not reveal incorrect Boolean expressions.
- It does not guarantee testing of all feasible execution paths.
- It does not ensure that input boundaries are tested.
- It may miss defects caused by combinations of conditions.

6. Improved Testing Strategies

- Branch Coverage: Execute both true and false outcomes of every decision.
- Condition Coverage: Make each individual Boolean condition evaluate to true and false.
- Path Testing: Test feasible execution paths through the program.
- Boundary Value Analysis: Test values immediately below, at, and above limits.
- MC/DC: Verify that each condition can independently affect the overall decision outcome.

Conclusion: Statement coverage alone is insufficient for complex logic. Combining branch, condition, path, and boundary testing provides stronger coverage and improves defect detection.

---

5. Age Validation – Equivalence Class Partitioning

Question:

A tester uses Equivalence Class Partitioning for an age-validation field with a valid range of 0–100 but defines only two classes: negative numbers and positive numbers. Analyze the errors, identify missing boundaries, and propose corrected partitions and test cases.

Answer:

1. Understanding the System

An age-validation field accepts ages from 0 to 100 inclusive. Inputs outside this range, non-integer values, and missing values must be handled according to the system specification.

2. System Decomposition

- Input Collection: Receives the age.
- Type Validation: Checks whether the input is numeric and in the required format.
- Range Validation: Checks whether age is between 0 and 100.
- Result Handling: Accepts valid ages or displays an error.

3. Errors in the Existing Partitioning

The original strategy incorrectly groups all positive numbers into one class. This class contains both valid ages and invalid values such as 101 and 150. It also fails to treat zero as a valid age and does not cover negative, non-numeric, or empty inputs separately.

4. Corrected Equivalence Classes

Class	Input Range	Classification
EC1	Age below 0	Invalid
EC2	Age from 0 to 100 inclusive	Valid

EC3	Age above 100	Invalid
EC4	Non-numeric input	Invalid
EC5	Empty input	Invalid if required

5. Boundary Value Test Cases

Test Case	Age	Expected Result
TC1	-1	Reject
TC2	0	Accept
TC3	1	Accept
TC4	99	Accept
TC5	100	Accept
TC6	101	Reject
TC7	"abc"	Reject
TC8	Empty input	Display validation error

6. Improved Testing Strategy

Select representative values from every equivalence class and test values around the lower and upper limits. Verify that the application accepts valid ages and rejects invalid inputs with appropriate error messages.

Conclusion: Correct ECP classification separates valid and invalid inputs, while BVA checks the range limits. Both techniques improve age-validation accuracy and reduce input-related defects.

---

6. Condition Coverage – Complex Boolean Expression

Question:

A developer claims full Condition Coverage for a complex if statement containing the expression if (A && B). Analyze the coverage claim, identify possible gaps in testing, and propose corrected test cases that verify condition coverage and decision coverage.

Answer:

1. Understanding the System

Condition Coverage checks whether each individual Boolean condition evaluates to both true and false. Decision Coverage checks whether the complete decision evaluates to both true and false.

For the expression A AND B, the overall result is true only when both A and B are true.

2. System Decomposition

- Condition A: Evaluates the first Boolean input.
- Condition B: Evaluates the second Boolean input.
- Decision Module: Combines the conditions using AND.
- True Branch: Executes when both conditions are true.
- False Branch: Executes when at least one condition is false.

3. Corrected Test Cases

Test Case	A	B	A AND B
TC1	True	True	True
TC2	True	False	False
TC3	False	True	False
TC4	False	False	False

4. Coverage Analysis

- TC1 and TC2 make B evaluate to both true and false while A remains true.
- TC1 and TC3 make A evaluate to both true and false while B remains true.
- TC1 covers the true decision outcome.
- TC2, TC3, and TC4 cover the false decision outcome.

Thus, the four combinations provide 100% condition and decision coverage for this expression.

5. Common Errors

Testing only (True, True) and (False, False) makes both conditions evaluate to true and false, and covers both decision outcomes. However, it does not demonstrate that each condition independently affects the decision result.

6. Improved Testing Strategy

Use the complete truth table when practical. For complex expressions, apply Modified Condition/Decision Coverage (MC/DC), which verifies that each condition independently changes the overall decision outcome while other conditions are appropriately controlled.

Conclusion: Condition and decision coverage measure different aspects of Boolean logic. Using truth tables and MC/DC where appropriate provides stronger evidence that complex conditions work correctly.

7. Discount System – Decision Table Testing

Question:

A QA team applies Decision Table Testing for a discount system but misses several combinations of Premium Member and Coupon Applied, leading to price calculation errors. Analyze the incomplete decision table and reconstruct an exhaustive table.

Answer:

1. Understanding the System

A discount system calculates the final product price based on whether a customer is a premium member and whether a valid coupon is applied. Missing combinations can cause incorrect discounts or inconsistent pricing.

2. System Decomposition

- Customer Validation: Identifies premium membership.
- Coupon Validation: Checks coupon validity.
- Discount Calculation: Determines applicable discounts.
- Price Calculation: Computes the final amount.
- Output Module: Displays the final price.

3. Errors in the Existing Decision Table

- Some combinations of membership and coupon status are missing.
- Discount rules may not be defined for every combination.
- Invalid coupons may be treated as valid.
- The final price may be calculated inconsistently.
- Combined discounts may be applied incorrectly.

4. Corrected Decision Table

Assume premium members receive a 10% discount and valid coupons provide an additional 5% discount. For this example, the discounts are additive and apply to the original price.

Rule	Premium Member	Valid Coupon Applied	Discount	Expected Result
R1	Yes	Yes	15%	Apply both discounts
R2	Yes	No	10%	Apply member discount
R3	No	Yes	5%	Apply coupon discount
R4	No	No	0%	No discount

5. Test Cases

Assume the original price is ₹1,000.

Test Case	Conditions	Expected Final Price
TC1	Premium member, valid coupon	₹850
TC2	Premium member, no coupon	₹900
TC3	Non-premium member, valid coupon	₹950
TC4	Non-premium member, no coupon	₹1,000

6. Improved Testing Strategy

Test all four combinations. Add tests for expired coupons, invalid coupon codes, zero-priced products, rounding, and refund calculations. If discounts are compounded rather than additive, update the expected results according to the business rules.

Conclusion: A complete decision table ensures that every combination of membership and coupon status is tested. This improves pricing accuracy and prevents discount-related production defects.

---

8. Loop Testing – Zero, Maximum, and Nested Iterations

Question:

During unit testing of a looping structure, a tester checks only the first and last iterations, n = 1 and n = max. Defects occurring at n = 0 and n = max + 1 are missed. Evaluate the errors, identify root causes, and recommend a corrected loop-testing approach.

Answer:

1. Understanding the System

Loop testing verifies that loops execute the correct number of times, terminate properly, and handle boundary conditions. Errors may occur when a loop executes zero times, exceeds its allowed limit, or contains nested loops.

2. System Decomposition

- Initialization: Sets the loop counter.
- Loop Condition: Determines whether iteration continues.
- Loop Body: Performs the required operation.
- Counter Update: Changes the iteration value.
- Termination: Stops the loop when the condition becomes false.

3. Errors in the Existing Approach

- Zero iterations are not tested.
- Values beyond the maximum are not tested.
- Off-by-one errors may remain undetected.
- Nested loops may contain untested combinations.
- Infinite loops and incorrect termination conditions may be missed.

4. Corrected Test Cases

Assume a loop permits a maximum of 10 iterations.

Test Case	Number of Iterations	Expected Result
TC1	0	Loop body is skipped
TC2	1	Executes once
TC3	9	Executes nine times
TC4	10	Executes maximum permitted iterations
TC5	11	Reject or handle according to specification
TC6	Negative count	Reject invalid input or skip safely

5. Testing Nested Loops

For nested loops, test the inner loop with zero, one, and multiple iterations while keeping the outer loop controlled. Then test the outer loop with zero, one, and multiple iterations. Finally, test selected combinations of both loops to verify termination and correct results.

6. Improved Testing Strategy

Use boundary-oriented loop testing, validate initialization and counter updates, and monitor termination. Apply structural coverage measures and test error handling for excessive iteration counts.

Conclusion: Testing only the first and last iterations is insufficient. Testing zero, one, typical, maximum, and over-limit counts helps detect off-by-one errors, incorrect termination, and loop-related defects.

---

9. Safety-Critical Braking System – Error Guessing, BVA, and State Transition Testing

Question:

An engineer uses Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the problems, evaluate the impact on test adequacy, and recommend a combination of BVA and State Transition Testing.

Answer:

1. Understanding the System

An electronic braking system monitors speed, brake-pedal input, wheel sensors, and operating conditions to activate the appropriate braking response. Incorrect behavior may cause accidents, so testing must be systematic and safety-focused.

2. System Decomposition

- Sensor Input: Receives wheel-speed and pedal data.
- Input Validation: Detects invalid or inconsistent readings.
- Control Logic: Determines the required braking response.
- Actuator Control: Sends commands to braking components.
- Fault Detection: Detects sensor and communication failures.
- Safety Response: Activates the specified safe behavior.

3. Problems with Error Guessing Alone

- Testing depends heavily on the tester's experience.
- Important boundaries may be overlooked.
- State transitions may remain untested.
- Test coverage cannot be systematically demonstrated.
- Rare combinations of sensor and operating conditions may be missed.

4. Boundary Value Analysis

Assume a hypothetical sensor input is permitted between 0 and 250 units.

Test Case	Sensor Input	Expected Result
TC1	-1	Detect invalid input
TC2	0	Process according to specification
TC3	1	Process valid input
TC4	249	Process valid input
TC5	250	Process maximum valid input
TC6	251	Detect out-of-range input

The actual sensor range and expected safety response must be defined by the system requirements.

5. State Transition Testing

Test transitions among the following conceptual states:

- Normal: Sensors and braking functions operate normally.

- Braking: Brake input triggers the required braking action.
- Fault Detected: An invalid sensor or communication failure is identified.
- Safe Mode: The system enters its specified safe operating state.

Verify normal-to-braking, normal-to-fault, fault-to-safe-mode, and recovery transitions where permitted. Test invalid transitions, delayed sensor responses, and repeated fault signals.

6. Improved Testing Strategy

Use BVA for sensor limits, State Transition Testing for operating modes, and Error Guessing as an additional technique. Add fault injection, integration testing, and requirements-based verification in a controlled environment. Confirm that the specified safety response occurs for each relevant failure.

Conclusion: Error Guessing alone is inadequate for a safety-critical system. Combining structured boundary and state testing improves coverage and supports more reliable safety assurance, although it does not by itself prove that the system is safe.

---

10. Control Flow Graph – Missing Else Branches

Question:

A tester designs a Control Flow Graph for a module but fails to represent the else branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors, identify the root causes, and propose a corrected CFG and independent paths.

Answer:

1. Understanding the System

A Control Flow Graph represents all possible execution flows through a program. Every conditional statement must include its true and false outcomes. Omitting else branches hides error paths and creates an incomplete testing model.

2. System Decomposition

- Input Validation: Checks input values.
- Authentication: Verifies user credentials.
- Authorization: Checks access permissions.
- Processing Module: Performs the requested operation.
- Error Handling: Handles invalid input and unauthorized access.
- Output Module: Displays success or error results.

3. Sample Program

```
def transfer(amount, balance, authorized):
    if authorized:
        if amount > 0:
            if amount <= balance:
                print("Transfer successful")
            else:
                print("Insufficient balance")
        else:
            print("Invalid amount")
    else:
        print("Unauthorized")
```

4. Corrected Control Flow Representation

Start → Check whether the user is authorized.

- If false, display "Unauthorized" and terminate.
- If true, check whether the amount is greater than zero.
- If false, display "Invalid amount" and terminate.
- If true, check whether the amount is within the available balance.
- If true, display "Transfer successful".
- If false, display "Insufficient balance".
- All terminal branches proceed to the End node.

The CFG must contain all three decision nodes, both outcomes of each decision, and the corresponding processing or error-handling nodes.

5. Independent Execution Paths

Path	Input Condition	Expected Result
P1	Authorized, positive amount, sufficient balance	Transfer successful
P2	Authorized, positive amount, insufficient balance	Insufficient balance
P3	Authorized, zero or negative amount	Invalid amount
P4	Unauthorized user	Unauthorized

6. Test Cases

Assume the account balance is ₹5,000.

Test Case	Authorized	Amount	Expected Result
TC1	Yes	₹1,000	Transfer successful
TC2	Yes	₹6,000	Insufficient balance

TC3	Yes	₹0	Invalid amount
TC4	Yes	-₹100	Invalid amount
TC5	No	₹1,000	Unauthorized

7. Root Causes and Improvements

The original CFG omitted false branches and error-handling nodes. As a result, tests concentrated on successful execution and did not cover rejection paths. Reconstructing the CFG with all decisions and branches enables systematic branch coverage. Boundary tests should also be added for amounts equal to the balance and values immediately above or below it.

Conclusion: A complete CFG must represent every decision outcome, including nested else branches. Testing all feasible paths improves branch coverage and helps detect authorization, validation, and balance-checking defects.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand